

REVISÃO DE CÓDIGO COM LLMs EM UMA PLATAFORMA DE GERENCIAMENTO DE ORDENS DE SERVIÇO

[ESPAÇO RESERVADO PARA AUTORIA APÓS O ACEITE]

RESUMO

A manutenção de sistemas desktop para pequenas empresas exige revisões periódicas para assegurar legibilidade, correção e estabilidade. Este estudo analisou a aplicação de um modelo de linguagem de larga escala (LLM) como assistente na revisão e refatoração do Orby, software em Python e PySide6 para gestão de ordens de serviço. Quatro módulos centrais (persistência, repositório, importação e interface) foram avaliados com prompts estruturados. Das 14 recomendações geradas pela LLM, 5 foram aceitas e implementadas, 3 adaptadas pela equipe e 6 rejeitadas após verificação e testes. A ferramenta agilizou a identificação de validações ausentes e trechos com alto acoplamento, mas também sugeriu alterações inadequadas no banco e regras de negócio. Conclui-se que a LLM atua com eficácia como segunda fonte de análise, desde que as decisões finais e a validação de comportamento permaneçam sob rigoroso controle humano.

Palavras-chave: Revisão de código; Refatoração; Manutenibilidade; Avaliação humana.

1. INTRODUÇÃO

Os modelos de linguagem de larga escala vêm sendo incorporados a diferentes atividades da engenharia de software, incluindo análise, programação, testes, documentação e manutenção. Seu emprego pode apoiar a produção de artefatos, mas as respostas precisam ser examinadas pela equipe responsável pelo projeto (HOU et al., 2024). A revisão de código permanece como etapa essencial para assegurar legibilidade, manutenibilidade e correção, onde desenvolvedores avaliam conformidade com regras de negócio e prevenção de regressões. Nesse cenário, a IA generativa atua como fonte complementar de inspeção, desde que nenhuma recomendação seja adotada sem validação e testes práticos.

O Orby é uma aplicação desktop desenvolvida em Python com PySide6 e SQLite, destinada ao controle operacional de assistências técnicas (clientes, equipamentos, ordens de serviço e importação CSV/XLSX). Por tratar de processos críticos — como integridade referencial, bloqueio de exclusões indevidas e consistência financeira —, o sistema demanda revisões criteriosas a cada incremento.

O objetivo geral deste estudo foi analisar a contribuição de uma LLM para a revisão e o refinamento supervisionado do código da plataforma Orby.

2. FUNDAMENTAÇÃO TEÓRICA

A revisão de código visa detectar defeitos e verificar padrões antes da integração à base principal. A refatoração busca aprimorar a estrutura interna sem alterar o comportamento observável externamente, exigindo validação contínua dos requisitos funcionais após cada alteração aceita.

Pesquisas recentes indicam que LLMs podem gerar comentários e propostas de refinamento de código, mas sua utilidade prática depende diretamente do contexto fornecido no prompt e da capacidade crítica do desenvolvedor em filtrar alucinações e respostas incorretas (LU et al., 2023; GUO et al., 2024; HAIDER et al., 2024). Hou et al. (2024) reforçam que o emprego de modelos sem verificação humana acarreta sérios riscos de introdução de defeitos.

3. MATERIAL E MÉTODOS

O estudo foi aplicado e descritivo, realizado sobre os módulos centrais do Orby: `database.py` (esquema e migrações SQLite), `repository.py` (acesso a dados e regras de negócio), `importer.py` (leitura CSV/XLSX) e `main_window.py` (interface PySide6). Utilizou-se prompt padronizado solicitando análise de legibilidade, acoplamento, validações e riscos de regressão. A Figura 1 apresenta o fluxo metodológico de revisão e validação adotado.

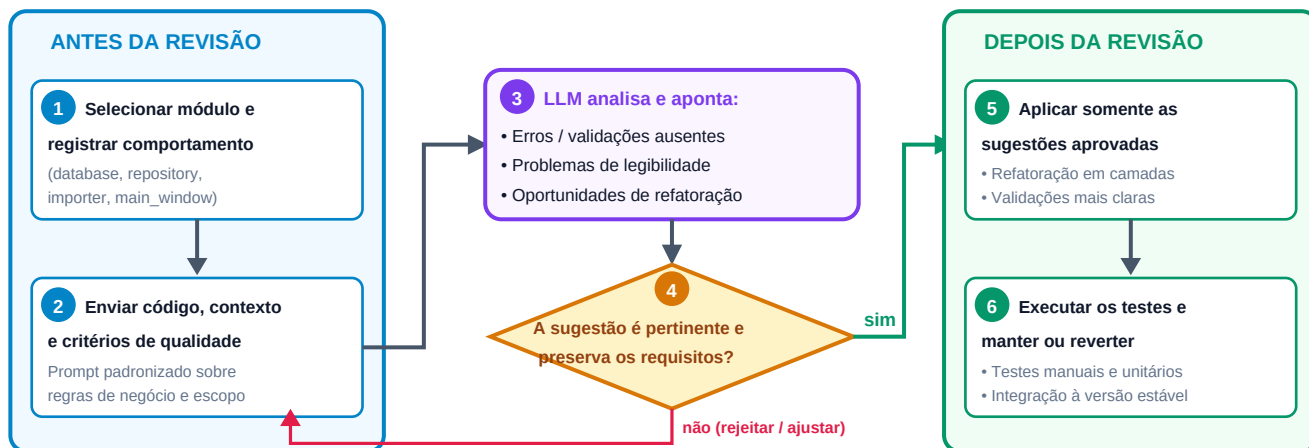


Figura 1: Fluxograma do processo de revisão e validação supervisionada por LLM.

4. RESULTADOS E DISCUSSÕES

Foram avaliadas 14 recomendações geradas pela LLM, das quais **5 foram aceitas**, **3 modificadas** e **6 rejeitadas**. A Tabela 1 sintetiza a distribuição das recomendações por tema.

Tema Identificado	Aceitas	Modificadas	Rejeitadas
Legibilidade e formatação da UI	2	1	0
Validação e regras de negócio	1	1	2
Arquitetura e desacoplamento	1	0	2
Persistência e ciclo de conexões	0	0	1
Testes automatizados	1	1	1

Tabela 1: Classificação das recomendações produzidas pela LLM.

Evidência de Refatoração: No módulo de repositório, a validação de OS foi aprimorada para conferir clareza à obrigatoriedade do equipamento cadastrado antes do encaminhamento a diagnóstico:

Antes (Validação implícita): if not equipment_id: raise ValueError('Selecione o equipamento da OS.')	Depois (Validação explícita e tipada): if equipment_id is None: raise ValueError('Selecione um equipamento cadastrado para a OS.')
---	---

A LLM destacou-se ao apontar linhas condensadas na interface gráfica e acoplamento entre diálogos. Contudo, produziu sugestões inadequadas, como adotar ORM externo desnecessário para o porte do MVP e alterar a consulta de duplicidade para operador AND, o que geraria duplicidades no banco. Os testes executados pela equipe garantiram que apenas sugestões seguras fossem incorporadas.

5. CONCLUSÕES

Conclui-se que a revisão de código assistida por LLM acelerou significativamente o diagnóstico de legibilidade e validações no Orby. No entanto, a tendência a gerar recomendações genéricas ou inconsistentes com as regras de negócio reafirma que a IA não substitui a inspeção humana e a execução de testes sistemáticos.

REFERÊNCIAS

GUO, Qi et al. Exploring the potential of ChatGPT in automated code refinement: an empirical study. In: **IEEE/ACM INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING**, 46., 2024. p. 390-402. DOI: 10.1145/3597503.3623306.

HAIDER, Md. Asif et al. Prompting and fine-tuning large language models for automated code review comment generation. **arXiv**, 2024. DOI: 10.48550/arXiv.2411.10129.

HOU, Xinyi et al. Large language models for software engineering: a systematic literature review. **ACM Transactions on Software Engineering and Methodology**, 2024. DOI: 10.1145/3695988.

LU, Junyi; YU, Lei; LI, Xiaojia; YANG, Li; ZUO, Chun. LLaMA-Reviewer: advancing code review automation with large language models through parameter-efficient fine-tuning. In: **IEEE INTERNATIONAL SYMPOSIUM ON SOFTWARE RELIABILITY ENGINEERING**, 34., 2023. p. 647-658. DOI: 10.1109/ISSRE59848.2023.00026.